

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы данных»
Тема: Платформа для игры в круговые шахматы

Студенты гр. 3342

Преподаватель

Галеев А.Д.

Романов Е.А.

Хайруллов Д.Л.

Заславский М. М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

Романов Е. А.

Галеев А. Д.

Хайруллов Д. Л.

Группа 3342

Тема работы: Веб-приложение для игры hotseat, хранения партий и тестирования ботов для круговых шахмат

Исходные данные:

Необходимо реализовать веб-приложение для игры в круговые шахматы.

Хранение партий, статистики, аккаунты, подключение ботов и их тестирование.

Содержание пояснительной записки:

«Введение»,

«Сценарии использования»,

«Модель данных»,

«Разработанное приложение»,

«Выводы»,

«Приложения»,

«Литература»

Предполагаемый объем пояснительной записки:

Не менее 15 страниц.

Дата выдачи задания: 10.02.2026

Дата сдачи реферата: 02.06.2026

Дата защиты реферата: 02.06.2026

Студенты гр. 3342

Романов Е. А.

Галеев А. Д.

Хайруллов Д. Л.

Преподаватель

Заславский М. М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение на одну из заданных тем, использующее нереляционную базу данных, в команде. Была выбрана тема создания платформы для игры в круговые шахматы. Приложение позволяет пользователям регистрироваться, играть партии друг с другом (в режиме hotseat) и против ботов, просматривать и искать сыгранные партии, вести статистику, а также управлять ботами и проводить их тестирование. Для хранения пользователей, ботов и партий используется документоориентированная СУБД MongoDB.

Исходный код хранится в репозитории проекта:
<https://github.com/moevm/nsql1h26-chess>

SUMMARY

This course involved developing an application on one of the given topics using a non-relational database as a team. The chosen topic was a platform for playing circular chess. The application allows users to register, play games against each other (hotseat mode) and against bots, browse and search played games, keep statistics, and also manage bots and run their testing. MongoDB, a document-oriented database, is used to store players, bots and games.

The source code is stored in the project repository:
<https://github.com/moevm/nsql1h26-chess>

СОДЕРЖАНИЕ

1. Введение	7
1.1. Актуальность решаемой проблемы	7
1.2. Постановка задачи	7
1.3. Предлагаемое решение.....	7
1.4. Качественные требования к решению	8
2. Сценарии использования	9
2.1. Макет UI	9
2.2. Сценарии использования для задачи	10
2. Представление данных.....	12
2.3. Вывод о преобладающих операциях	14
3. Модель данных	15
3.1. Нереляционная модель данных (MongoDB)	15
3.1.1 Графическое представление	15
3.1.2 Описание назначений коллекций, типов данных и сущностей	15
3.1.3 Оценка удельного объема информации, хранимой в модели	18
3.2. Аналог модели данных для SQL СУБД (PostgreSQL)	20
3.2.1 Графическое представление модели.....	20
3.2.2 Описание назначений коллекций, типов данных и сущностей	20
3.2.3. Оценка удельного объёма информации	25
3.3. Сравнение моделей.....	27
3.3.1 Удельный объём информации.....	27
3.3.2 Запросы по отдельным сценариям использования.....	27
3.3.3 Вывод	28
4. Разработанное приложение	30
4.1. Краткое описание	30
4.2. Используемые технологии	30
4.3. Снимки экрана приложения	30
5. Выводы	34
5.1 Достигнутые результаты.....	34
5.2 Недостатки и пути для улучшения полученного решения.....	34
5.3 Будущее развитие решения	34

6. Приложения.....	36
6.1. Документация по сборке и развёртыванию	36
6.2. Инструкция для пользователя	36

1. Введение

1.1. Актуальность решаемой проблемы

Шахматы остаются одной из самых популярных игр в мире. Круговые шахматы - один из таких вариантов в котором доска и расположение фигур меняют привычную геометрию игры, сохраняя при этом основные принципы движения фигур.

С развитием онлайн-сервисов растёт спрос на платформы, которые позволяют не только играть, но и сохранять историю партий, анализировать их и соревноваться с компьютерными соперниками. Отдельный интерес представляет возможность разработки и автоматического тестирования игровых ботов: чтобы оценить силу алгоритма, требуется проводить большое количество партий и хранить их результаты. Это порождает потребность в приложении, которое объединяет в себе игровой движок, хранилище партий и инструменты анализа статистики.

1.2. Постановка задачи

Необходимо реализовать приложение для игры hotseat в круговые шахматы с хранением партий, пользователей и ботов в базе данных. Приложение должно поддерживать регистрацию и авторизацию пользователей, проведение партий между пользователями и против ботов, просмотр и поиск сыгранных партий, ведение статистики, управление ботами, а также массовый импорт и экспорт данных.

1.3. Предлагаемое решение

Предлагается реализовать веб-приложение, использующее СУБД MongoDB. Партия как единица данных естественным образом представляется в виде документа, который содержит список ходов, ссылки на участников, итоговый результат и историю смены статусов. Пользователи и боты хранятся в единой

коллекции с полем-дискриминатором, что упрощает работу со статистикой и историей статусов. Такой подход позволяет получать всю информацию о партии или участнике одним запросом.

1.4. Качественные требования к решению

К решению предъявляются следующие качественные требования:

- поддержка регистрации и авторизации пользователей;
- проведение партий в режиме «пользователь против пользователя» (hotseat) и «пользователь против бота»;
- сохранение полной истории ходов каждой партии и возможность её просмотра в режиме реплея;
- поиск и фильтрация партий и пользователей с пагинацией и сортировкой;
- ведение статистики участников (победы, поражения, ничьи);
- создание, редактирование и автоматическое тестирование ботов;
- ведение истории изменения статусов сущностей;
- массовый импорт и экспорт данных.

2. Сценарии использования

2.1. Макет UI

На изображении представлен макет UI с возможными переходами между страницами приложения (см. рис. 1).

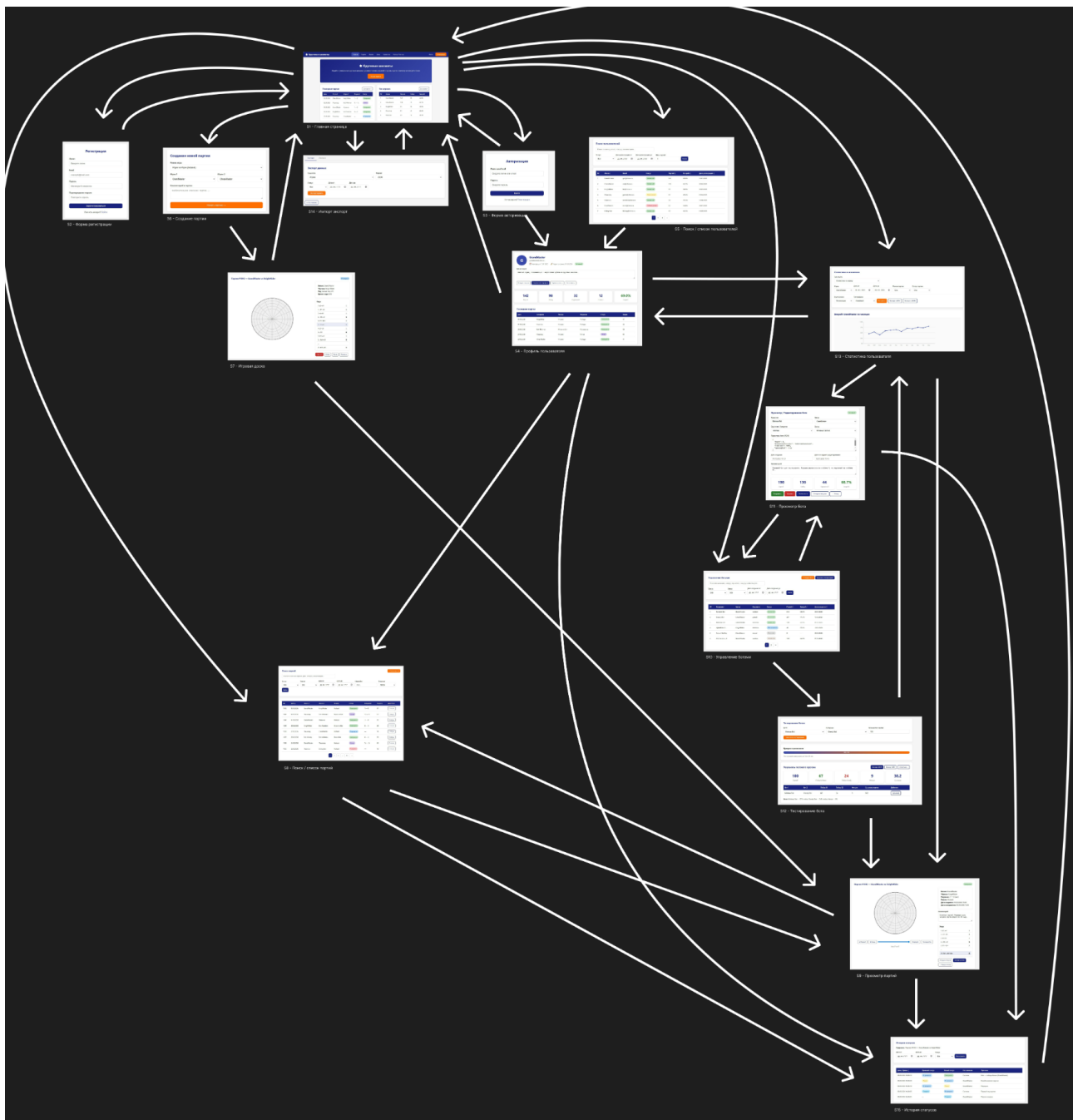


Рисунок 1 - все экраны приложения

2.2. Сценарии использования для задачи

1. Импорт данных

Массовый импорт данных из файла.

Предусловия: у пользователя есть файл с данными в формате JSON, он находится на главной странице.

Основной сценарий:

Таблица 1 - Основной сценарий

Шаг	Пользователь	Система
1	Переходит на страницу импорта/экспорта и выбирает вкладку «Импорт»	Отображает страницу импорта
2	Загружает файл	Показывает сообщение об успешной загрузке
3	Выбирает стратегию при конфликтах (пропустить, перезаписать, добавить)	Принимает выбранную стратегию
4	Нажимает «Импортировать»	Загружает данные и выводит лог обработанных записей

Альтернативный сценарий:

Таблица 2 - Альтернативный сценарий

Шаг	Пользователь	Система
2	Загружает повреждённый файл или файл неверного формата	Показывает сообщение об ошибке

Шаг	Пользователь	Система
3	Импортирует файл с частично некорректными строками	Выполняет частичный импорт, лог содержит список ошибочных строк

Создание бота

Предусловия: пользователь авторизован и находится на странице управления ботами.

Основной сценарий:

Таблица 3 - основной сценарий

Шаг	Пользователь	Система
1	Нажимает «Создать бота»	Открывает экран редактирования бота
2	Вводит название, стратегию, параметры и комментарий	Валидирует данные
3	Нажимает «Сохранить»	Создаёт бота со статусом «draft» и записывает историю статусов

Альтернативный сценарий:

Таблица 4 - Альтернативный сценарий

Шаг	Пользователь	Система
3	Сохраняет бота с уже существующим названием	Показывает предупреждение о дубликате

2. Представление данных

Поиск и просмотр партий.

Предусловия: в базе есть сохранённые партии, пользователь находится на главной странице.

Основной сценарий:

Таблица 5 - основной сценарий

Шаг	Пользователь	Система
1	Нажимает кнопку «Партии»	Открывает экран поиска партий
2	Вводит критерии (статус, режим, даты) и применяет фильтры	Принимает параметры и выполняет поиск
3	Просматривает результаты	Отображает таблицу партий с пагинацией и сортировкой
4	Кликает на строку с партией	Открывает экран просмотра с доской и навигацией по ходам

Альтернативный сценарий:

Таблица 6 - Альтернативный сценарий

Шаг	Пользователь	Система
3	Ни одна партия не найдена	Отображает пустую таблицу с соответствующим сообщением

Помимо просмотра партий в табличном виде, пользователь может открыть конкретную партию в режиме и пошагово проигрывать ходы на игровой доске, что является основной формой визуального представления данных об истории игрового процесса в приложении.

3. Анализ данных

Просмотр профиля и статистики.

Предусловия: пользователь авторизован, существуют участники со сыгранными партиями.

Основной сценарий:

Таблица 7 - основной сценарий

Шаг	Пользователь	Система
1	Нажимает кнопку «Пользователи» и вводит критерии поиска	Отображает таблицу результатов
2	Кликает на нужного пользователя	Открывает профиль с личными данными и статистикой
3	Просматривает статистику и список последних партий	Отображает агрегированные показатели (победы, поражения, ничьи)
4	При необходимости переходит к истории статусов	Отображает лог изменений статуса сущности

4. Экспорт данных

Массовый экспорт данных

Предусловия: в базе есть данные для экспорта.

Основной сценарий:

Таблица 8 - основной сценарий

Шаг	Пользователь	Система
1	Переходит на страницу импорта/экспорта, вкладка «Экспорт»	Отображает форму экспорта
4	Нажимает «Экспортировать»	Формирует файл и отдаёт его на скачивание

2.3. Вывод о преобладающих операциях

В приложении преобладают операции записи, связанные с проведением партий: каждая партия порождает создание документа, добавление множества ходов и обновление статистики участников, при этом операции чтения также частые, но, как правило, обслуживаются одним запросом за счёт хранения связанных данных внутри одного документа. Таким образом, нагрузка смешанная, однако генерация и сохранение партий делают запись значимой составляющей.

3. Модель данных

3.1. Нереляционная модель данных (MongoDB)

3.1.1 Графическое представление

Модель представлена на рисунке ниже (см. рис. 2).

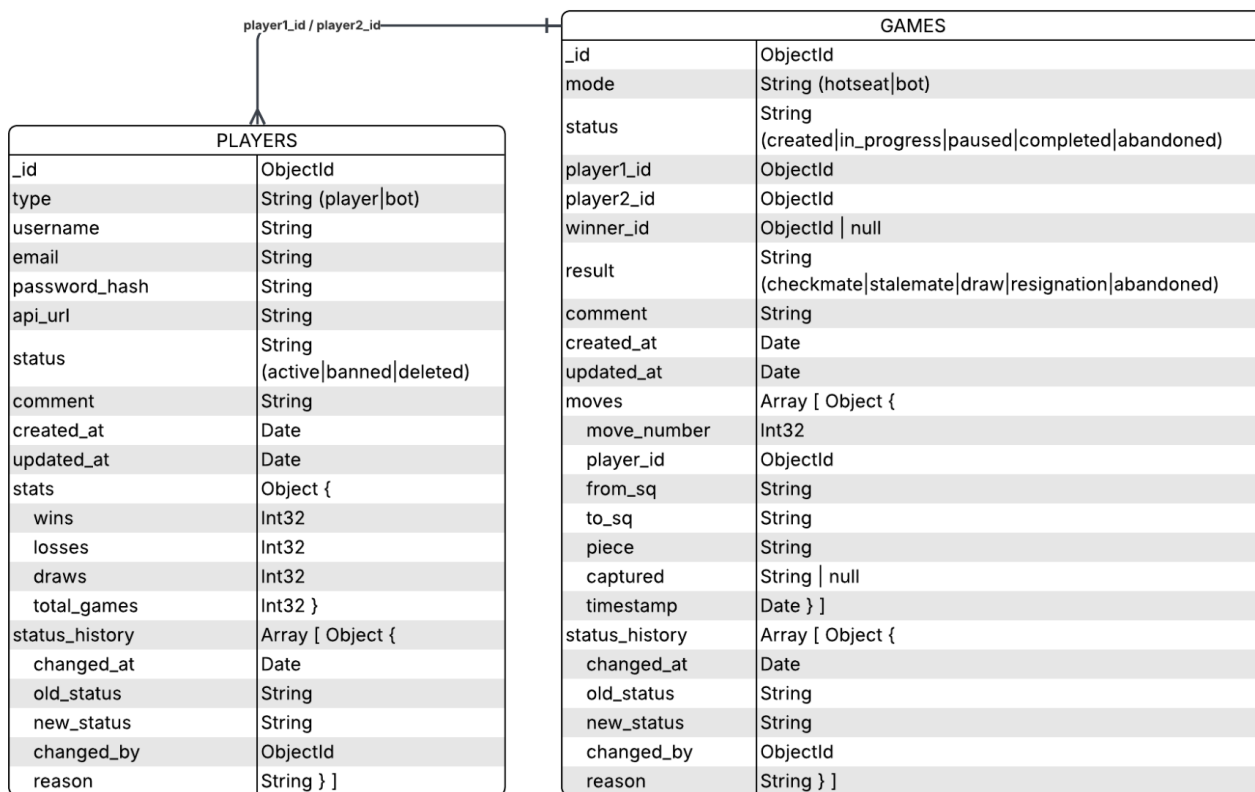


Рисунок 2 - графическое представление нереляционной базы данных

3.1.2 Описание назначений коллекций, типов данных и сущностей

Модель данных MongoDB включает две коллекции: **players** и **games**. Пользователи и боты хранятся в единой коллекции **players** с полем **type**, что устраняет необходимость в отдельной коллекции для ботов. Связи между коллекциями реализуются через хранение **ObjectId**-ссылок. Статистика и история статусов встроены в родительские документы, а ходы партии хранятся как массив поддокументов внутри документа партии.

Коллекция players. Хранит учётные записи всех участников - пользователей (type = «player») и ботов (type = «bot») в единой коллекции. Объединение оправдано высоким сходством атрибутов: у обоих типов есть статус, статистика и история изменения статусов. Обязательные поля для каждого типа закреплены через механизм Schema Validation MongoDB. Основные поля коллекции приведены в таблице.

Таблица 9 - Поля коллекции Players

Поле	Тип BSON	Размер, байт	Назначение
_id	ObjectId	12	Уникальный идентификатор участника
type	String	~6	Тип участника: «player» или «bot»
username	String	~30	Логин / отображаемое имя
email	String	~50	Электронная почта (для player)
password_hash	String	120	Хеш пароля bcrypt (для player)
api_url	String	~80	Адрес API бота (для bot)
status	String	~12	Текущий статус участника

Поле	Тип BSON	Размер, байт	Назначение
comment	String	~100	Комментарий / описание бота
created_at / updated_at	Date	8 + 8	Даты создания и обновления
stats. {wins,losses,draws,total_games}	Int32	4 × 4	Статистика участника
status_history[]	Array	-	История смены статусов (поддокументы)

Коллекция games. Хранит партии с полным списком ходов и историей статусов; используется в том числе для тестовых партий между ботами (mode = «bot»). Поля player1_id и player2_id ссылаются на документы коллекции players; тип участника определяется по полю type соответствующего документа.

Таблица 10 - Поля коллекции Games

Поле	Тип BSON	Размер, байт	Назначение
_id	ObjectId	12	Идентификатор партии
mode	String	~14	Режим игры (hotseat, bot)
status	String	~20	Текущий статус партии
player1_id / player2_id	ObjectId	12 + 12	Ссылки на участников
winner_id	ObjectId/null	12 / 0	Победитель

Поле	Тип BSON	Размер, байт	Назначение
result	String	~20	Результат (checkmate, stalemate, ...)
comment	String	~100	Комментарий к партии
created_at / updated_at	Date	8 + 8	Даты создания и обновления
moves[]	Array	-	Список ходов (поддокументы)
status_history[]	Array	-	История смены статусов

3.1.3 Оценка удельного объема информации, хранимой в модели

Размеры документов оцениваются в формате BSON, где каждое поле хранится как: 1 байт типа + имя ключа + завершающий нулевой байт + значение; строковые значения дополнительно содержат 4 байта длины и нулевой байт. Введём обозначения и модельные допущения:

- N - количество партий (основная переменная);
- $P = N / 20$ - число игроков (в среднем 20 партий на игрока);
- $B = 10$ - число ботов (относительно постоянная величина);
- $M = 40$ - среднее число ходов в партии;
- $S_p = 2, S_g = 3, S_b = 3$ - среднее число записей истории статусов у игрока, партии и бота.

С учётом служебных байтов BSON получены следующие размеры документов:

- документ игрока (type = player): $527 + 21 + S_p \times 163 = 874$ байта;

- документ бота (type = bot): $416 + 21 + S_b \times 163 = 926$ байт;
- документ партии: $338 + (12 + M \times 141) + (21 + S_g \times 163) = 6500$ байт (при $M = 40$).

Общий объём модели складывается из P документов игроков, B документов ботов и N документов партий:

$$V_mongo = P \times 874 + B \times 926 + N \times 6500$$

После подстановки $P = N/20$ и $B = 10$ получаем линейную зависимость от числа партий:

$$V_mongo \approx 6544 \cdot N + 9260 \text{ (байт)}$$

При $N = 1\,000$ партий объём составляет ≈ 6.5 МБ, при $N = 100\,000$ - ≈ 654 МБ. Основной вклад в рост вносит коллекция games: каждая партия добавляет ≈ 6.5 КБ, тогда как игроки и боты дают пренебрежимо малую долю. Дополнительно документ партии растёт линейно по числу ходов M (по 141 байту на ход).

3.2. Аналог модели данных для SQL СУБД (PostgreSQL)

3.2.1 Графическое представление модели

Модель представлена на рисунке ниже (см. рис. 3).

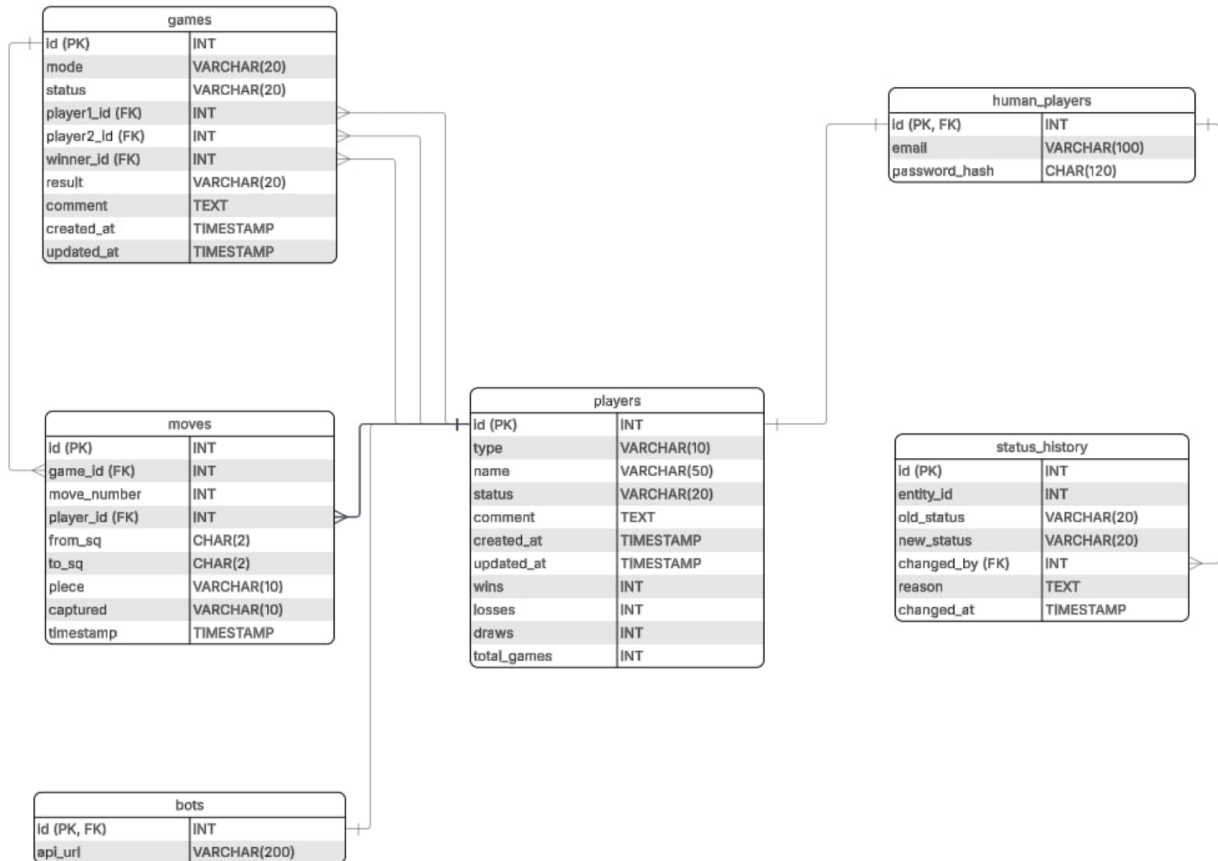


Рисунок 3 - графическое представление реляционной базы данных

3.2.2 Описание назначений коллекций, типов данных и сущностей

Реляционный аналог состоит из 6 таблиц (см. рис. 3). Встроенные поддокументы MongoDB (stats, moves, status_history) вынесены в отдельные таблицы. Для воспроизведения полиморфизма участников применяется наследование: общая таблица players со всеми общими атрибутами и две специализирующие таблицы - human_players и bots (с PK = FK на players.id).

База данных состоит из шести таблиц. Встроенные поддокументы MongoDB (статистика, ходы, история статусов) вынесены в отдельные таблицы; полиморфизм участников реализован наследованием - общая таблица `players` и две специализирующие таблицы `human_players` и `bots`, у которых первичный ключ одновременно является внешним ключом на `players.id`. Сводная информация о таблицах приведена ниже, а далее по каждой таблице дан побайтовый разбор полей.

Таблица `players`. Базовая таблица всех участников, содержит атрибуты, общие для пользователей и ботов, включая денормализованную статистику. Специфичные поля вынесены в наследующие таблицы `human_players` и `bots`.

Таблица 11 - Поля коллекции `players`

Поле	Тип	Размер, байт	Назначение
<code>id (PK)</code>	INT	4	Уникальный идентификатор участника
<code>type</code>	VARCHAR(10)	≈ 14	Тип участника: «player» или «bot»
<code>name</code>	VARCHAR(50)	≈ 54	Отображаемое имя участника
<code>status</code>	VARCHAR(20)	≈ 24	Текущий статус участника
<code>comment</code>	TEXT	≈ 100	Комментарий / описание
<code>created_at</code>	TIMESTAMP	8	Дата создания записи
<code>updated_at</code>	TIMESTAMP	8	Дата последнего обновления
<code>wins</code>	INT	4	Число побед

Поле	Тип	Размер, байт	Назначение
losses	INT	4	Число поражений
draws	INT	4	Число ничьих
total_games	INT	4	Всего сыграно партий

Итого с учётом служебных данных строки — ≈ 252 байта; число строк равно числу участников ($N/20$ пользователей и 10 ботов).

Таблица human_players. Специализирующая таблица участников-людей. Первичный ключ id одновременно является внешним ключом на players.id, что реализует наследование «один к одному».

Таблица 12 - Поля коллекции human_players

Поле	Тип	Размер, байт	Назначение
id (PK, FK)	INT	4	Идентификатор; ссылка на players.id
email	VARCHAR(100)	≈ 104	Электронная почта пользователя
password_hash	CHAR(120)	120	Хеш пароля (bcrypt)

Итого ≈ 252 байта на строку; число строк равно числу пользователей ($\approx N/20$).

Таблица bots. Специализирующая таблица участников-ботов. Первичный ключ id также является внешним ключом на players.id.

Таблица 13 - Поля коллекции bots

Поле	Тип	Размер, байт	Назначение
id (PK, FK)	INT	4	Идентификатор; ссылка на players.id
api_url	VARCHAR(200)	≈ 204	Адрес внешнего API бота

Итого ≈ 232 байта на строку; число строк равно числу ботов ($B \approx 10$).

Таблица games. Таблица партий со ссылками на участников и победителя. В отличие от документной модели, ходы хранятся не внутри партии, а в отдельной таблице moves.

Таблица 14 - Поля коллекции games

Поле	Тип	Размер, байт	Назначение
id (PK)	INT	4	Идентификатор партии
mode	VARCHAR(20)	≈ 24	Режим игры (hotseat, bot)
status	VARCHAR(20)	≈ 24	Текущий статус партии
player1_id (FK)	INT	4	Ссылка на первого участника
player2_id (FK)	INT	4	Ссылка на второго участника
winner_id (FK)	INT	4	Ссылка на победителя (может быть NULL)
result	VARCHAR(20)	≈ 24	Результат (checkmate, stalemate, ...)

Поле	Тип	Размер, байт	Назначение
comment	TEXT	≈ 100	Комментарий к партии
created_at	TIMESTAMP	8	Дата создания
updated_at	TIMESTAMP	8	Дата обновления

Итого ≈ 228 байт на строку; число строк равно числу партий N .

Таблица moves. Таблица ходов: каждая строка соответствует одному ходу некоторой партии. Самая объёмная по числу строк таблица модели.

Таблица 15 - Поля коллекции moves

Поле	Тип	Размер, байт	Назначение
id (PK)	INT	4	Идентификатор хода
game_id (FK)	INT	4	Ссылка на партию (games.id)
move_number	INT	4	Порядковый номер хода
player_id (FK)	INT	4	Кто сделал ход (players.id)
from_sq	CHAR(2)	2	Клетка-источник
to_sq	CHAR(2)	2	Клетка-назначение
piece	VARCHAR(10)	≈ 14	Тип фигуры
captured	VARCHAR(10)	≈ 14	Взятая фигура (если есть)
timestamp	TIMESTAMP	8	Время хода

Итого ≈ 82 байта на строку; число строк равно суммарному числу ходов $M \cdot N$ (при $M = 40$ — $40 \cdot N$).

Таблица status_history. Универсальная таблица истории смены статусов для любой сущности (партии, пользователя или бота). В документной модели этой таблице соответствует встроенный массив status_history внутри родительского документа.

Таблица 16 - Поля коллекции status_history

Поле	Тип	Размер, байт	Назначение
id (PK)	INT	4	Идентификатор записи
entity_id	INT	4	Ссылка на сущность, чей статус изменился
old_status	VARCHAR(20)	≈ 24	Предыдущий статус
new_status	VARCHAR(20)	≈ 24	Новый статус
changed_by (FK)	INT	4	Кто изменил статус (players.id)
reason	TEXT	≈ 100	Причина / комментарий изменения
changed_at	TIMESTAMP	8	Время изменения

Итого ≈ 192 байта на строку; число строк в среднем составляет $\approx 3.1 \cdot N + 30$.

3.2.3. Оценка удельного объёма информации

Суммируя объёмы всех таблиц с теми же константами (N — число партий, $P = N/20$, $B = 10$, $M = 40$), получаем:

$$V_{\text{sql}} = (N/20+10) \cdot 252 + (N/20) \cdot 252 + 10 \cdot 232 + N \cdot 228 + 40N \cdot 82 + (3.1N+30) \cdot 192$$

$$V_{\text{sql}} \approx 4128 \cdot N + 10600 \text{ (байт)}$$

При $N = 1\,000$ партий объём составляет ≈ 4.1 МБ, при $N = 100\,000$ — ≈ 413 МБ. Наибольший вклад в рост вносит таблица moves: каждая партия порождает одну строку в games и M строк в moves. Зависимость, как и в документной модели, линейная по N .

3.3. Сравнение моделей

3.3.1 Удельный объём информации

Сравним объёмы обеих моделей при равном числе партий N .

Таблица 17 – Сравнение объёмов моделей

Параметр	MongoDB	PostgreSQL
Формула объёма, байт	$\approx 6544 \cdot N + 9260$	$\approx 3568 \cdot N + 6860$
Объём при $N = 1\,000$	≈ 6.5 МБ	≈ 3.6 МБ
Объём при $N = 10\,000$	≈ 65.4 МБ	≈ 35.7 МБ
Объём при $N = 100\,000$	≈ 654 МБ	≈ 357 МБ
Избыточность	$< +0.1\%$	$\approx +18\%$

Асимптотический коэффициент SQL-модели примерно вдвое меньше, чем у MongoDB. Основная причина - формат BSON хранит имена полей в каждом документе, тогда как в реляционной модели схема задаётся один раз. При этом сама SQL-модель более избыточна относительно минимально необходимого объёма ($\approx +18\%$) за счёт ключей `id` и внешних ключей `game_id` в таблице `moves` и `entity_id` в `status_history` - полей, которые в документной модели заменяются вложенностью. Обе модели растут линейно ($O(N)$).

3.3.2 Запросы по отдельным сценариям использования

Сравнение числа запросов и задействованных коллекций/таблиц по основным сценариям приведено в таблице.

Таблица 18 – Сравнение числа запросов

Сценарий	Mongo, запр.	SQL, запр.	Mongo, колл.	SQL, табл.
Регистрация	3	5	1	3

Сценарий	Mongo, запр.	SQL, запр.	Mongo, колл.	SQL, табл.
Авторизация	1	1	1	2
Партия (hotseat / бот)	4 + M	6 + M	2	4
Поиск/просмотр партий	2	3	1	3
Профиль и статистика	3	3	2	2
Управление ботами	2	3–4	1	3
Тестирование ботов	$G_t + 5$	$5 + 3 \cdot G_t$	2	5
Импорт	1	3	1	3
Экспорт	1	1	1	2
История статусов	1	1	1	1

MongoDB требует меньше запросов в большинстве сценариев и задействует меньше коллекций (1–2 против 1–5 таблиц), поскольку связанные данные (stats, moves, status_history) хранятся внутри основного документа. Разница особенно заметна в сценариях, зависящих от M и G_t : при $M = 40$ для проведения партии MongoDB выполняет 44 запроса против 46 у SQL, а при $G_t = 10$ для тестирования ботов — 15 против 35.

3.3.3 Вывод

Для поставленной задачи модель MongoDB предпочтительнее. Партия как единица данных естественно ложится в документ — ходы, статистика и история статусов читаются и записываются одним запросом без обращения к нескольким таблицам. Несмотря на большой удельный объём (за счёт хранения имён полей в каждом документе), документная модель обеспечивает меньшее число запросов

и меньшую сложность кода для ключевых сценариев приложения, что важнее при преобладании операций создания и чтения партий целиком.

4. Разработанное приложение

4.1. Краткое описание

Реализовано веб-приложение для игры в круговые шахматы. Система построена по клиент-серверной архитектуре и состоит из трёх контейнеризованных сервисов, которые поднимаются совместно через docker-compose:

- **frontend** - клиентская часть, отвечающая за пользовательский интерфейс, игровую доску и взаимодействие со страницами поиска, профилей, ботов и импорта/экспорта;
- **backend** - серверная часть, предоставляющая REST API: проведение партий, валидация ходов, CRUD-операции над пользователями и ботами, поиск и фильтрация, тестирование ботов, импорт и экспорт данных, аутентификация и авторизация;
- **database** - экземпляр MongoDB, хранящий коллекции `players` и `games`.

Клиент обращается к серверу по HTTP, сервер выполняет бизнес-логику и сохраняет данные в MongoDB. Изоляция сервисов в отдельных контейнерах упрощает развёртывание и обеспечивает воспроизводимость окружения.

4.2. Используемые технологии

- **Frontend** - JavaScript (без фреймворка), HTML и CSS.
- **Backend** - Node.js с использованием фреймворка Express, реализующий REST API.
- **База данных** – MongoDB.
- **Контейнеризация и оркестрация** — Docker и docker-compose; всё приложение собирается и запускается одной командой.

4.3. Снимки экрана приложения

Ниже приведены снимки экрана работающего приложения (см. рис. 4 - 6).

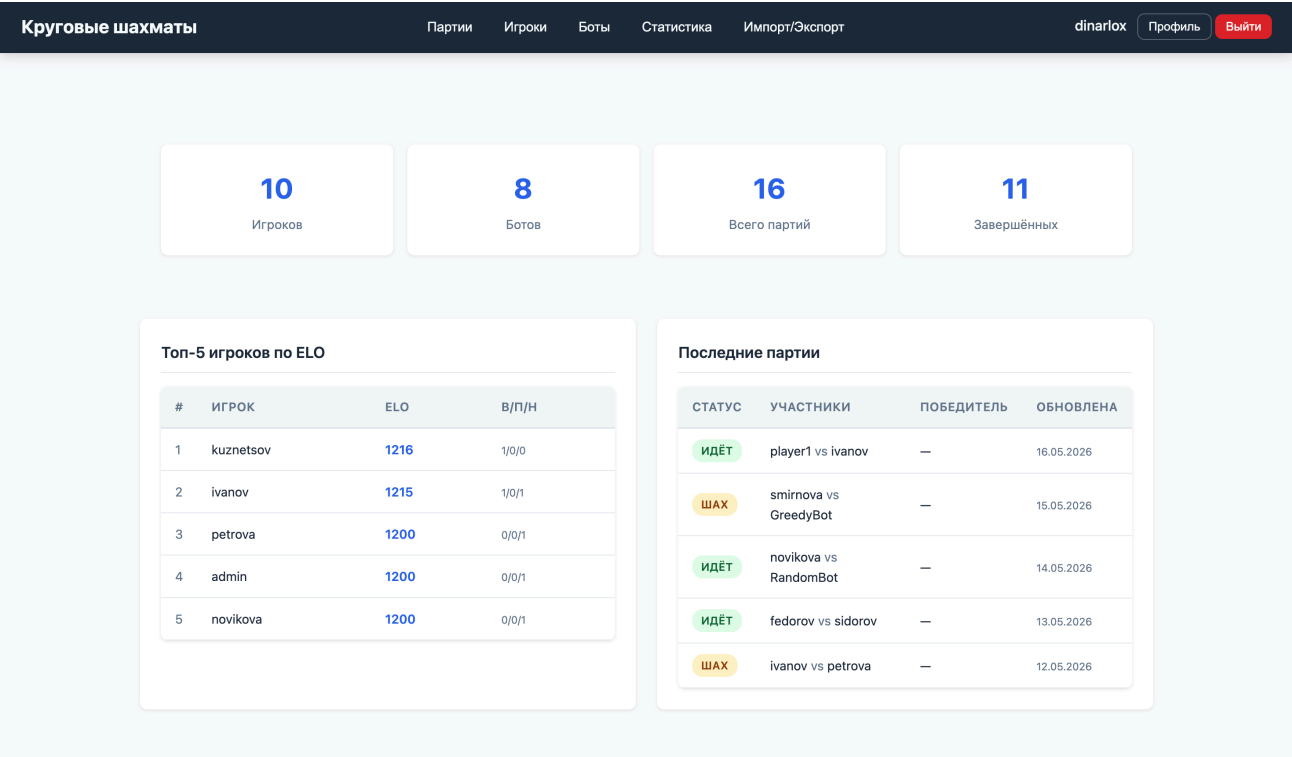


Рисунок 4 - главная страница приложения

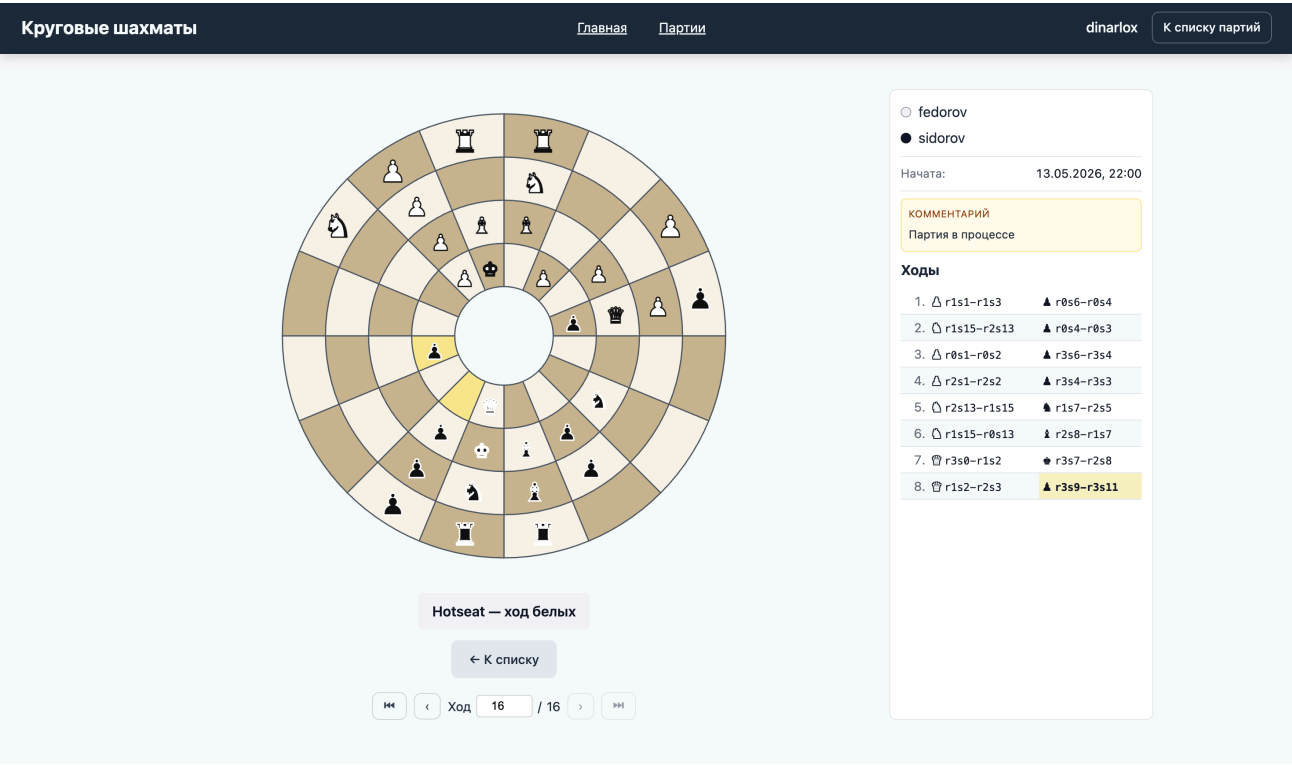


Рисунок 5 - игровая доска

Круговые шахматы

Партии

Игроки

Боты

Статистика

Импорт/Экспорт

dinarlox

Профиль

Выйти

Партии

+ Новая партия

Фильтры

СТАТУС

Все

УЧАСТНИК (ИМЯ)

Поиск по имени...

КОММЕНТАРИЙ

Поиск в комментариях...

КОЛ-ВО ХОДОВ

От

До

ДАТА СОЗДАНИЯ (ОТ)

dd . mm . yyyy

ДАТА СОЗДАНИЯ (ДО)

dd . mm . yyyy

ДАТА ОБНОВЛЕНИЯ (ОТ)

dd . mm . yyyy

ДАТА ОБНОВЛЕНИЯ (ДО)

dd . mm . yyyy

Применить

Сбросить

СТАТУС	БЕЛЫЕ	ЧЁРНЫЕ	ХОДОВ	РЕЗУЛЬТАТ	ОЧЕРЕДЬ	СОЗДАНА	ОБНОВЛЕНА
ИДЁТ	player1	ivanov	12	—	Белые	16.05.2026, 13:00	16.05.2026, 13:22
ШАХ	smirnova	GreedyBot	61	—	Чёрные	15.05.2026, 17:00	15.05.2026, 17:18
ИДЁТ	novikova	RandomBot	9	—	Чёрные	14.05.2026, 15:00	14.05.2026, 15:12
ИДЁТ	fedorov	sidorov	16	—	Белые	13.05.2026, 22:00	13.05.2026, 22:20
ШАХ	ivanov	petrova	16	—	Белые	12.05.2026, 18:00	12.05.2026, 18:25

Рисунок 6 - поиск и список партий

Круговые шахматы

Партии

Игроки

Боты

Статистика

Импорт/Экспорт

admin

Профиль

Выйти

Импорт / Экспорт данных

Экспорт

Скачать все данные приложения (игроки, боты, партии) в формате JSON.

Экспортировать всё

Импорт

Загрузить данные из JSON-файла, полученного при экспорте.

Стратегия при конфликтах

Пропустить существующие

Файл JSON

Choose file No file chosen

Импортировать всё

Рисунок 7 - Импорт и экспорт данных

32

Круговые шахматы

Партии

Игроки

Боты

Статистика

Импорт/Экспорт

admin

Профиль

Выйти

← Игроки / novikova

N

novikova

АКТИВЕН

novikova@chess.ru

Зарегистрирован: 26.04.2026, 15:00

Обновлён: 02.06.2026, 23:04

Новикова Ольга

0

ПОБЕДЫ

0

ПОРАЖЕНИЯ

1

НИЧЬИ

1

ВСЕГО ПАРТИЙ

1200

ELO (ЛЮБИТЕЛЬ)

История статусов

⇒ Редактировать

Партии игрока

Фильтры

СТАТУС	ЦВЕТ	СОПЕРНИК	РЕЗУЛЬТАТ	ELO	ХОДОВ	ОБНОВЛЕНА
ИДЁТ	Белые	RandomBot	—	—	9	14.05.2026, 15:12
НИЧЬЯ	Белые	admin	Ничья	1200 (0)	40	08.05.2026, 21:05

Всего: 2

Рисунок 8 – Профиль игрока

5. Выводы

5.1 Достигнутые результаты

В результате работы реализовано веб-приложение для игры в круговые шахматы с хранением данных в нереляционной СУБД MongoDB. Приложение поддерживает регистрацию и авторизацию пользователей, проведение партий в режиме «пользователь против пользователя» (hotseat) и «пользователь против бота», сохранение полной истории ходов каждой партии и её просмотр в режиме реплея. Реализованы поиск и фильтрация партий и участников с пагинацией и сортировкой, ведение статистики (победы, поражения, ничьи), создание, редактирование и автоматическое тестирование ботов, а также ведение истории смены статусов сущностей. Поддерживается массовый импорт и экспорт данных. Спроектирована и обоснована документоориентированная модель данных: проведено её сравнение с реляционным аналогом по удельному объёму и по числу запросов на ключевых сценариях, что подтвердило целесообразность выбора MongoDB для данной задачи.

5.2 Недостатки и пути для улучшения полученного решения

Несмотря на то что заявленный функционал реализован в полном объёме, решение имеет ряд ограничений. Хранение всех ходов внутри одного документа партии ограничено максимальным размером BSON-документа (16 МБ), что для очень длинных серий ходов теоретически может стать проблемой. Кроме того, нет возможности редактировать права пользователей, например, назначать других администраторов. Для улучшения стоит добавить более развитую систему прав и доработать модель хранения данных.

5.3 Будущее развитие решения

В дальнейшем приложение может быть расширено сетевыми партиями между пользователями в реальном времени с синхронизацией состояния доски,

автоматического подбора соперников, проведением турниров и круговых чемпионатов ботов. Также возможна расширенная визуализация статистики (графики активности, распределение результатов и длительности партий).

6. Приложения

6.1. Документация по сборке и развёртыванию

Для сборки и запуска приложения необходимо выполнить следующие действия:

1. Клонировать репозиторий проекта.
2. Перейти в директорию проекта.
3. Из корня проекта выполнить сборку и запуск через docker-compose.
`docker compose up --no-cache`

После запуска приложение будет доступно по адресу <http://localhost:8080>.

6.2. Инструкция для пользователя

Базовый сценарий работы с приложением выглядит следующим образом:

- открыть приложение в браузере и при необходимости зарегистрироваться или войти;
- для начала игры нажать «Новая партия», выбрать режим (против пользователя в режиме hotseat или против бота) и начать партию; ходы делаются на игровой доске, система валидирует их и обновляет состояние;
- для просмотра сыгранных партий перейти в раздел «Партии», задать фильтры и открыть нужную партию в режиме реплея;
- для просмотра статистики перейти в раздел «Пользователи» и открыть профиль участника;
- для работы с ботами перейти в раздел «Боты», где можно создать или отредактировать бота, а также запустить их автоматическое тестирование;
- для массовой загрузки или выгрузки данных использовать раздел «Импорт/Экспорт»

7. Литература

1. Документация MongoDB.

<https://www.mongodb.com/docs/manual/>

2. Документация Node.js.

<https://nodejs.org/docs/latest/api/>

3. Документация Express.

<https://expressjs.com/>

4. Методика оценки размера модели данных.

http://se.moevm.info/doku.php/staff:courses:no_sql_introduction:calculating_data_model_size

5. Репозиторий проекта.

<https://github.com/moevm/nsql1h26-chess>